Advanced Programming Technologies (Java)

Individual Project

THE LAUGHTER LEGACY

The Legacy of Laughter



ADVENTURE PROJECT ATLAS

2026

Report Contents

01 / Project Aim / 02

02 / Why One Piece / 03

The Laugh as a Legacy / 04

03 / Visual Style / 05

04 / Project Structure / 06

5.1 Class Hierarchy / 07

5.2 Interfaces / 08

5.3 Base Class Laugh / 09

5.4 BattleLaugh - Battle Laughter / 12

5.5 DreadLaugh - Fear Laughter / 13

5.6 CruelLaugh - Cruel Laughter (final class) / 14

5.7 MusicalLaugh - Musical Laughter / 15

5.8 ForcedSmileLaugh - Forced Smile / 16

5.9 InheritedLaugh - Inherited Laughter / 17

5.10 Main Class / 18

5.11 Meeting the Requirements / 20

5.12 Fixed Errors / 21

06 / Conclusions / 22

01 / Project Aim

Project aim: to practise the basics of object-oriented programming in Java by making our own project. During this work, we need to:

build a hierarchy of seven classes with an abstract base class and an abstract method;

describe four interfaces and connect them to the classes in the hierarchy;

override random settings, keyboard input, file saving and screen output in every child class;

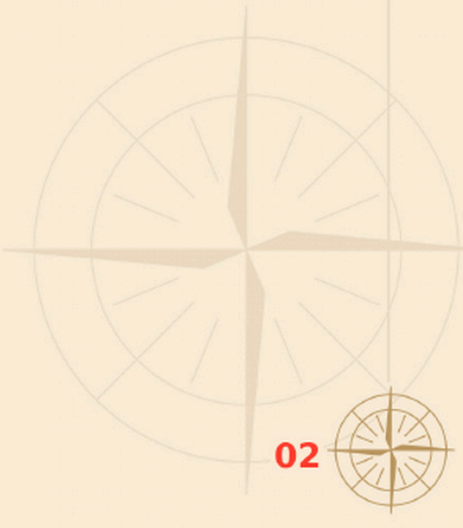
use the **final** and **static** modifiers, the **instanceof** operator and type casting;

create arrays in **main** with objects of the base class and interface types, and call the special methods of each class;

prepare a report that explains the idea and structure of the program.

The program should not only meet the task requirements. It should also work as a small living model: the characters' laughter affects each other and changes their morale.

Objects affect each other: attacks, fear, music and memory change morale.



02 / Why One Piece

Why One Piece

One Piece is a story about freedom, friendship and dreams. It has many colourful characters. We remember each hero not only by appearance and strength, but also by the way they act. This world is useful for an object-oriented model: heroes belong to natural “families” (fighters, musicians, and people linked to curses and memories). Each hero has common features and special features. This gives us an almost ready inheritance hierarchy.



Fig. 1. Different heroes - one common base class.

Why Laughter

Laughter in **One Piece** is a special part of the story. We know Luffy by “Shishishi”, Kaido by the loud “Worororo”, Teach by “Zehahaha”, Brook by “Yohohoho”, and Robin by the quiet “Fufufu”. Laughter shows a character’s nature: joy, danger, mockery, music and memories of the past.

The Laugh as a Legacy

Laughter is also an important part of the story. The name of the last island is **Laugh Tale** , which means “A Tale of Laughter”. Gol D. Roger smiled when he met his death. This is why the project is called **The Laughter Legacy** , or “The Legacy of Laughter”. The word “legacy” matches the main idea of OOP: inheritance. A character can get laughter from an ancestor, pass it to a friend or inherit it from a teacher.

Laughter is also easy to describe with data. It has an owner, a sound, a set of volume waves (an array), morale (a number that can go up and down) and the ability to affect other objects. This gives us a good study model with fields, methods, arrays and actions between objects.



Fig. 2. Gol D. Roger. Laughter as a legacy.

03 / Visual Style

The project uses an adventure manga style and a sea theme:

Naming. The classes are named after types of laughter (BattleLaugh, DreadLaugh, CruelLaugh, MusicalLaugh, ForcedSmileLaugh and InheritedLaugh). The example values come from the world of One Piece: Luffy, Kaido, Teach, Brook and Robin. Random names come from the crew: Nami, Zoro, Usopp, Sanji, Chopper and Franky.

Sounds. Laughter is written in a manga style: “SHISHISHI!”, “WORORORO!”, “ZEHAHAHAHA!”, “YOHOHOHO!” and “FUFUFU!”.

Game logic. Morale changes from -100 to 100, like a battle spirit scale. A hit lowers the target’s morale. Music and memories can raise it. Fear is stronger against an opponent who is already weak.

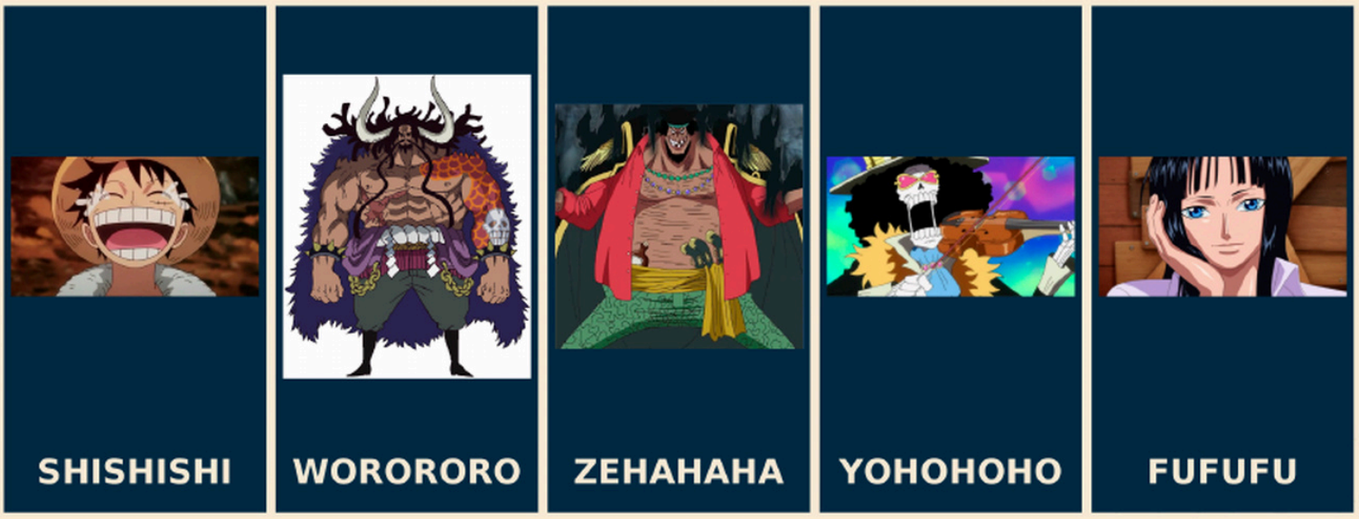


Fig. 3. Each type of laughter has its own class.

04 / Project Structure

The project is divided into four packages:

laughter.types - the base class Laugh and its child classes BattleLaugh, ForcedSmileLaugh and InheritedLaugh;

laughter.characters - the children of battle laughter: DreadLaugh, CruelLaugh and MusicalLaugh;

laughter.interfaces - four interfaces;

laughter.app - the Main class, which is outside the hierarchy.

Four packages - four roles

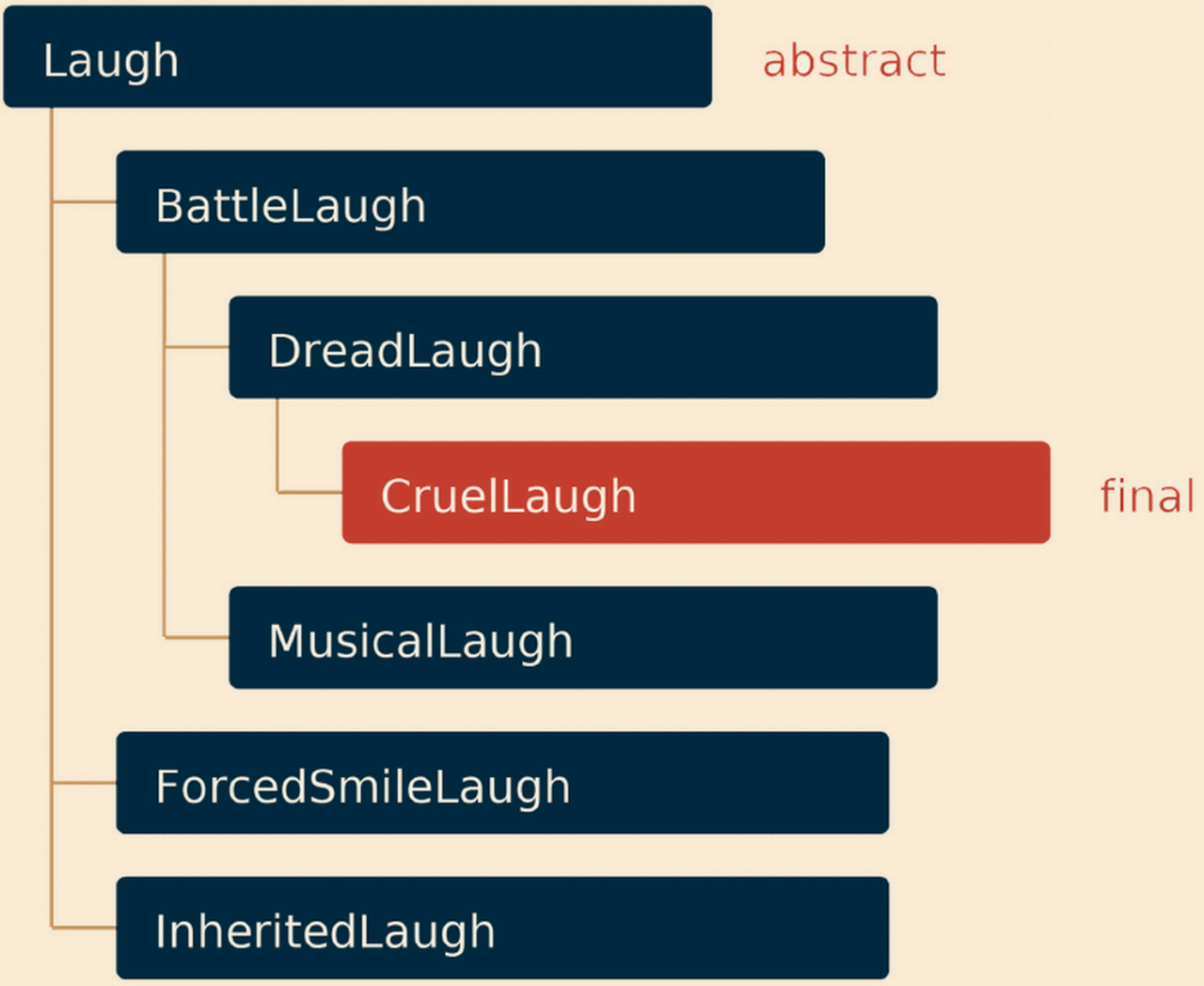
types - the base of the model and the direct children of Laugh.

characters - the next classes in the battle branch.

interfaces - contracts for abilities.

app - starts the program and shows how it works.

05 / Work Process
5.1 Class Hierarchy



There are seven classes in the hierarchy and four interfaces. The Main class is not part of the hierarchy.

Fig. 4. Class inheritance. Laugh is abstract; CruelLaugh is final. The lines connect each parent class to its child class.

The characters are examples for the objects and for the values of their fields. Inheritance is organised by types of laughter.

5.2 Interfaces

Each interface describes one ability and returns a number that shows the result of an action.

LaughPower - method int activate(Laugh target): a battle attack with laughter. It returns the morale points taken from the target.

Intimidating - method int frighten(Laugh target): fear. It returns the morale points taken from the target.

Transmittable - method int transmitTo(Laugh target): sends a mood to another character. It returns the morale points sent.

SoulResonance - method int resonate(Laugh target, int echo): uses stored Echo. It returns the amount of Echo used.

The interfaces are connected in this way: LaughPower is used by BattleLaugh (and also by DreadLaugh, CruelLaugh and MusicalLaugh). Intimidating is used by DreadLaugh. Transmittable and SoulResonance are used by MusicalLaugh and InheritedLaugh.

5.3 Base Class Laugh

Prototype: the common idea of laughter for any hero.



Fig. 5. Luffy. Basic laughter.

Fields

ownerName - the name of the owner of the laughter;

laughSound - the sound of the laughter (for example, “HAHAHA!”);

numberOfSoundWaves - the number of sound waves (long type);

volumeOfEachWave - an array with the volume of each wave;

morale - a number from -100 to 100;

wavesUsed - how many waves have already been released;

count - a private static field. It counts created objects (static);

MAX_WAVES - a constant with the value 1000, the maximum number of waves (final static);

RANDOM_NAMES and **RANDOM_SOUNDS** - private constant arrays for random names and sounds.

Laugh / Constructors

Laugh() - the default constructor. It sets “Unknown”, an empty wave array and morale 0. It calls randomizeOwn(), which fills the object with random values.

Laugh(String ownerName, String laughSound, long numberOfSoundWaves) - a constructor with parameters. If the number of waves is outside 0..1000, the array stays empty. Otherwise, it is created and filled with random volume values.

Laugh(Laugh other) - a copy constructor. It creates a separate copy, including the wave array.

All three constructors increase count. They do not have the public modifier, so they can be called only inside the package. This is a planned limit: objects are created through child classes.

Methods and work with waves

getOwnerName(), getLaughSound(), getNumberOfSoundWaves(), getMorale(), getWavesUsed() - return the field values.

getCount() - a static method. It returns the number of created laughs.

getWave(int index) - returns the volume of one wave. For a wrong index, it returns 0.

setWaves(double[] waves) - copies the given array, updates the number of waves and resets the used-wave counter. An array that is too long is ignored.

getVolumeOfEachWave() - returns a copy of the array, so outside code cannot change the original data (encapsulation).

getWavesLeft() - returns how many waves are left.

releaseWave() - releases the next wave and returns its volume.

restoreWaves() - restores all waves.

Laugh / Behaviour and Input

fillWaves() - a private method. It fills the waves with random volume values from 30 to 100.

randomizeOwn() - a private method. It chooses a random name, sound, 3 to 10 waves and morale from -20 to 20.

randomize() - a public method that calls randomizeOwn(). Child classes override it.

changeMorale(int amount) - changes morale and keeps it between -100 and 100.

readInt(Scanner sc, String text, int min, int max) - a static method. It reads an integer, checks the range and asks again after an error (static).

input(Scanner sc) - reads all fields from the keyboard.

save(PrintWriter out) - writes the fields to a file. The first line has the real class name, using getClass().getSimpleName().

printCommon() - a protected method that prints the common fields. It is declared protected final, so child classes cannot override it.

print() - an abstract method. Each child class decides how to show itself.

5.4 BattleLaugh - Battle Laughter

Prototype: Luffy, whose laughter comes together with a hit.



Fig. 6. Luffy. BattleLaugh.

Unique field: impactPower - the power of the hit.

Constructors: the default constructor calls super() and sets a random hit power from 20 to 80. The constructor with parameters (ownerName, laughSound, numberOfSoundWaves, impactPower) calls super(...) with three parameters.

getImpactPower() - returns the hit power.

randomizeOwn() - a private method. It sets the hit power.

randomize() - is overridden. First it calls super.randomize(), then it changes its own field.

print() - prints the common fields with printCommon() and the hit power.

input(Scanner) and **save(PrintWriter)** - are overridden. They call the parent method and add the class field.

chargeUp(int amount) - a special method. It increases the hit power. The value stays between 0 and 100.

activate(Laugh target) - the LaughPower method. Damage is one fifth of the hit power. If morale is 50 or higher, damage becomes 1.5 times stronger. The target loses this morale, and the attacker gets half of the damage as confidence.

5.5 DreadLaugh - Fear Laughter

Prototype: Kaido with his loud, thunder-like laugh.



Fig. 7. Kaido. DreadLaugh.

It inherits from BattleLaugh and implements Intimidating.

Unique field: fearLevel - the level of fear.

Constructors: the default constructor calls super() and sets a random fear level from 10 to 30. The constructor with parameters (ownerName, laughSound, numberOfSoundWaves, impactPower, fearLevel) calls the parent constructor with four parameters.

getFearLevel() , private **randomizeOwn()** , and the overridden **randomize()** , **input()** , **save()** and **print()** work in the same way: first the parent method runs, then the child field is handled.

frighten(Laugh target) - the Intimidating method. Its power is equal to fearLevel. If the target's morale is already negative, the power becomes 1.5 times stronger. The target loses this morale.

5.6 CruelLaugh - Cruel Laughter (final class)

Prototype: Marshall D. Teach and his “Zehahaha”.



Fig. 8. Marshall D. Teach. CruelLaugh.

It inherits from DreadLaugh. The class is declared **final** , so no other class can inherit from it.

Unique field: crueltyLevel - the level of cruelty.

Constructors: the default constructor calls super() and sets random cruelty from 10 to 40. The constructor with parameters (ownerName, laughSound, numberOfSoundWaves, impactPower, fearLevel, crueltyLevel) calls the parent constructor with five parameters.

getCrueltyLevel() and **setCrueltyLevel(int)** - give access to the field.

randomizeOwn() , **randomize()** , **input()** , **save()** and **print()** - use the usual override chain.

mock(Laugh target) - a special method for mockery. The target loses morale equal to the cruelty level, plus one quarter more if the target’s morale is already negative. The character who mocks gets one tenth of the cruelty level.

5.7 MusicalLaugh - Musical Laughter

Prototype: Brook and his “Yohohoho”.



Рис. 9. Брук со скрипкой. MusicalLaugh.

Fig. 9. Brook with his violin. MusicalLaugh.

It inherits from BattleLaugh and implements Transmittable and SoulResonance.

Unique field: tempoBpm - the tempo in beats per minute.

Constructors: the default constructor calls super() and sets a random tempo from 60 to 240. The constructor with parameters (ownerName, laughSound, numberOfSoundWaves, impactPower, tempoBpm) calls super() with four parameters.

getTempoBpm() , private **randomizeOwn()** , and the overridden **randomize()** , **input()** (range 0..300), **save()** and **print()** .

resonate(Laugh target, int echo) - a musical effect. The target gets morale equal to the tempo divided by 10 (not more than 30). The effect is shown on the screen. The method returns the amount of Echo used. If there is no Echo, it returns 0.

transmitTo(Laugh target) - gives the target morale equal to the tempo divided by 20.

5.8 ForcedSmileLaugh - Forced Smile

Prototype: a hero who laughs against their will, even when they feel something very different inside.
другое.



Fig. 10. Toko. ForcedSmileLaugh.

It inherits directly from Laugh.

Unique fields: trueEmotion - the real emotion (sad, happy or angry); curseLevel - the power of the curse that forces the smile.

Constructors: the default constructor calls super(), chooses an emotion at random and sets the curse level from 10 to 90. The constructor with parameters (ownerName, laughSound, numberOfSoundWaves, trueEmotion, curseLevel) calls the parent constructor with three parameters and applies the emotion.

applyEmotion() - a private method. It sets morale by emotion: sad = -70, happy = 70, angry = -40, otherwise 0.

randomizeOwn() , **randomize()** , **input()** (asks again until the value is correct), **save()** and **print()** are overridden.

revealEmotion() - a special method. The hero shows the real feeling, morale rises by 15, the emotion is returned and printed.

tryToStop() - a special method that tries to stop the laughter. It works if morale + 100 is not less than twice the curse level. If it works, morale rises by 10.

5.9 InheritedLaugh - Inherited Laughter

Prototype: Robin, who keeps the memory of those who came before her.



Fig. 11. Nico Robin. InheritedLaugh.

It inherits from Laugh and implements Transmittable and SoulResonance.

Unique fields: sourceLaugh - the laugh it was inherited from (a reference to a Laugh object); memoryStrength - the power of memory.

Constructors: the default constructor calls super(), sets source to null and chooses a random memory strength from 10 to 90. The constructor with parameters (ownerName, laughSound, numberOfSoundWaves, memoryStrength) calls the parent constructor with three parameters.

getSourceLaugh() and **getMemoryStrength()** - give access to the fields.

inheritFrom(Laugh source) - a special method. It copies the laugh sound and wave array from the source and remembers the source. A character cannot inherit from itself.

transmitTo(Laugh target) - gives the target morale equal to one fifth of memory strength. Without a source, it does nothing.

resonate(Laugh target, int echo) - needs a source and at least 2 Echo. The effect is half of memory strength plus three times Echo. The method prints a memory and returns 1 (one Echo used).

randomizeOwn() , **randomize()** , **input()** , **save()** (also writes the source) and **print()** are overridden.

5.10 Main Class

Main is outside the hierarchy and contains the only main method. It shows the task requirements step by step.

Creating objects. Six objects are created with constructors with parameters (Luffy, Kaido, Teach, Brook, Toko and Robin). Two more use default constructors (a random CruelLaugh and a BattleLaugh). This calls parent constructors both with and without parameters.

Base class array. All eight objects are placed in Laugh[] all. This is an example of upcasting.

Inheritance of laughter. robin.inheritFrom(luffy) - Robin takes Luffy's laughter.

Interface array. First, the program counts how many objects are LaughPower (using instanceof). Then it creates a LaughPower[] fighters array of the correct size and fills it by casting: (LaughPower) all[i].

General information. The program prints the static counter Laugh.getCount() and the constant Laugh.MAX_WAVES.

Object interaction

The program goes through all objects. It calls print() for each one, so the correct class version works (polymorphism). Then an instanceof chain finds the real type and uses a cast to call special methods: mock, frighten, resonate, transmitTo, chargeUp, activate, revealEmotion, tryToStop, inheritFrom and resonate. The most specific child class (CruelLaugh) must be checked first. Otherwise, the parent branch will run.

Using an interface. Each element in LaughPower[] attacks Kaido through activate. The result depends on the real class.

Keyboard input. The user can choose to fill the random CruelLaugh by hand through the overridden input method.

Saving to a file. All objects are saved one by one to laughs.txt with save. A write error is handled with try/catch.

Main / Saving the Result

A part of the laughs.txt file for the Teach object:

laughs.txt / Teach

```
Type: CruelLaugh
Owner: Teach
Laugh: ZEHAHAHAHA!
Waves: 5
Volumes: 82 86 64 53 86
Waves left: 5
Morale: 7
Impact power: 55
Fear level: 15
Cruelty level: 28
```

Fig. 12. Part of the laughs.txt file: saved data for the Teach object (CruelLaugh).

5.11 Meeting the Requirements

A hierarchy of seven classes. The base class Laugh is abstract, and the print() method is abstract.

The Main class has the main method and is outside the hierarchy.

Four interfaces are used. Each is connected to at least one class.

Each child class has its own fields and methods (see sections 5.4-5.9).

Parent constructors are called both with and without parameters.

Normal parent methods (changeMorale, printCommon, getOwnerName and others) are called from child classes and from main.

randomize() is overridden in every class, and constructors set the fields to random values by default.

Keyboard input input() is overridden in every class.

File saving save() is overridden in every class.

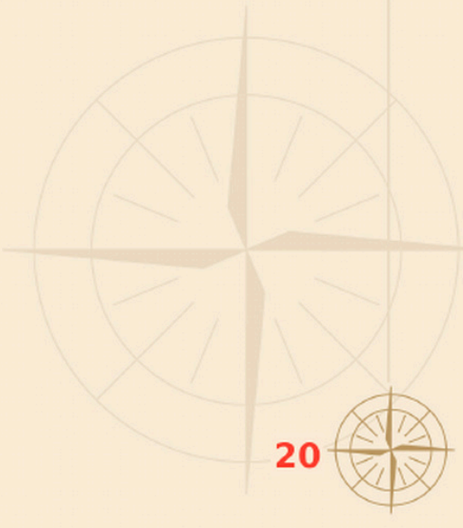
final: the MAX_WAVES variable, the printCommon() method and the CruelLaugh class.

static: the count variable and the getCount() and readInt() methods.

instanceof and type casting are used in main.

Arrays of the base class type (Laugh[]) and the interface type (LaughPower[]) are created in main.

The special methods of the classes are called in main, and their results are shown on the screen.



5.12 Fixed Errors

During testing, we found an error in the CruelLaugh class: it used the parent field fearLevel instead of its own field crueltyLevel. Because of this, the random cruelty value was always 0. During input, fear was asked for twice. In the screen output and the file, fear was repeated and cruelty was missing.

The error was fixed. Now random settings, input, output and saving use the class's own field. The repeated frighten method was also removed.

CruelLaugh -> crueltyLevel

The class now uses its own cruelty level when it creates random values, reads input, prints data and saves the file.

06 / Conclusions

During this work, we created a model of laughter from One Piece. It follows all the task requirements.

Inheritance. It helped us avoid repeated code. Common data and behaviour are stored in Laugh, and each child class adds only its own features. The chain Laugh -> BattleLaugh -> DreadLaugh -> CruelLaugh shows how a class gets new fields step by step.

Abstraction. The abstract Laugh class sets a common rule (print()), but we cannot create it directly. There is no “laughter in general”; there is the laughter of a specific hero.

Polymorphism. One call to all[i].print() or fighters[i].activate(kaido) gives different behaviour, depending on the real class.

Interfaces. They separate abilities (attack, frighten, pass a mood and use resonance) from the class hierarchy. Classes from different branches, for example MusicalLaugh and InheritedLaugh, can do the same thing but have different structures.

Encapsulation. Fields are private or protected, the wave array is returned as a copy, and the base class constructors are limited to the package.

The Next Port

final and static showed where changes must be forbidden and where data can be shared by the whole class.

instanceof and type casting are needed to get special methods from a general list. It is important to check the lowest class in the hierarchy first.

The main practical result is a better understanding of how to organise a program around objects and how to use general types (a base class and interfaces) to manage different objects in the same way. The project can be expanded: we can add new heroes, new abilities or a game mode without rewriting the existing code.



Fig. 13. The Straw Hat crew on their ship. End of the work. The next port: new child classes.